

SmartERP

One-page repo summary

Evidence source: code, package manifests, compose files, and env templates

What It Is

SmartERP is a TypeScript monorepo for an ERP platform with a NestJS backend, a React/Vite web app, a React Native Expo mobile app, and a shared package.

Repo evidence shows modules for inventory, sales, purchasing, HR, accounting, manufacturing, projects, notifications, workflows, search, reporting, and integrations.

Who It Is For

Primary user/persona: Not found in repo.

Role evidence in code includes admin, manager, production_manager, and production_user.

What It Does

- Protected dashboard for products, inventory, customers, suppliers, sales orders, and purchase orders.
- Accounting, HR, project, ecommerce, and manufacturing areas including BOMs, work orders, and work centers.
- Notifications center, notification preferences, audit logs, reports, search, settings, and tenant management.
- Web offline-first flows backed by IndexedDB/Dexie with auto-sync and retry logic when connectivity returns.
- Mobile offline cache and sync queue for products, inventory, and orders, plus secure token handling.
- Mobile device features for barcode or QR scanning, biometric login, and push notifications.

How It Works

- Web app: React 18 + Vite + Redux Toolkit + React Query + Ant Design. Protected routes live under /dashboard and the app restores sessions via /auth/refresh.
- Backend: NestJS monolith with auth, tenant, user, inventory, sales, purchasing, HR, accounting, manufacturing, project, dashboard, document, notification, report, search, workflow, and integration modules.
- Data/services: TypeORM uses PostgreSQL; global cache uses Redis; middleware adds metrics, logging, and tenant context; Swagger is exposed at /api/docs; monitoring initializes Sentry.
- Mobile: Expo + React Navigation + Redux. Offline API reads cache when offline and queues changes; sync pushes pending changes then pulls fresh server data. Local storage uses AsyncStorage and SQLite tables.

How To Run

1. Copy .env.example to .env.
2. Start the stack: docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build
3. Open the web app at http://localhost:5173
4. Backend listens on http://localhost:3000
5. API docs: http://localhost:3000/api/docs